Data Structure Project #3(실습)

2026 년 11 월 13 일

Due date: 2026 년 12 월 04 일 금요일 23:59:59 까지

본 프로젝트에서는 그래프 관련 알고리즘을 수행하는 프로그램을 구현한다. 이 프로그램은 그래프 정보가 저장된 텍스트 파일을 읽어 그래프를 생성하고, 명령어가 저장된 텍스트 파일을 읽어 BFS, DFS, Kruskal, Dijkstra, Bellman-Ford, Floyd 알고리즘과 정점에 인접한 간선을 힙 정렬(Heap Sort) 또는 병합 정렬(Merge Sort)로 정렬하는 연산을 수행한다. 주어지는 그래프 데이터는 기본적으로 방향성(direction)과 가중치(weight)를 모두 가지고 있으며, 그래프 데이터 형태에 따라 인접 리스트(adjacency list)와 인접 행렬(adjacency matrix)로 저장되어 있다. 하지만, 알고리즘의 특성에 따라 방향성 및 가중치의 적용 여부가 달라질 수 있다. Figure 1 은 그래프 연산 프로그램의 구조이다. 그래프 저장 형태에 따른 그래프 구축 방법과 각 알고리즘에 대한 설명은 program implementation 에서 설명한다.

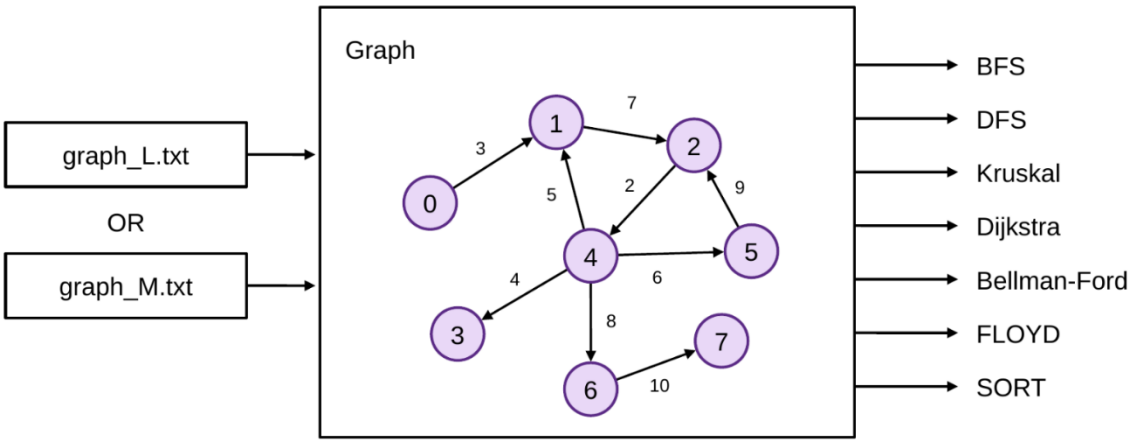


Figure 1. 그래프 연산 프로그램 구조

☐ Program implementation

1) 그래프 데이터

- 프로그램은 그래프 저장 유형, 정점 개수, 간선 연결 정보 및 가중치가 저장된 파일 graph_L.txt 또는 graph_M.txt 를 LOAD 명령어로 읽어 그래프를 생성한다.
- 두 파일 모두 첫 번째 줄에는 그래프 저장 유형(L: List, M: Matrix), 두 번째 줄에는 정점 개수 N 이 저장되어 있다. 이후 데이터는 그래프 저장 유형에 따라 구분된다
- 정점 개수 N 은 5 이상 100 이하이며, 정점 번호는 0 부터 N–1 까지 연속된 정수로 구성된다.
- 간선의 가중치는 –1,000,000 이상 1,000,000 이하의 정수이며, 가중치가 0 인 간선은 없다고 가정한다. 자기 자신으로 연결되는 간선과 동일한 출발 정점과 도착 정점을 갖는 중복 간선은 허용하지 않는다.
- 인접 리스트(adjacency list)는 **Figure 2** 와 같이 출발 정점을 나타내는 줄과 해당 정점에서 연결되는 도착 정점 및 가중치를 나타내는 줄로 구성된다. 출발 정점은 0 부터 N–1 까지 순서대로 모두 기재하며, 나가는 간선이 없는 정점은 출발 정점을 나타내는 줄만 존재한다.

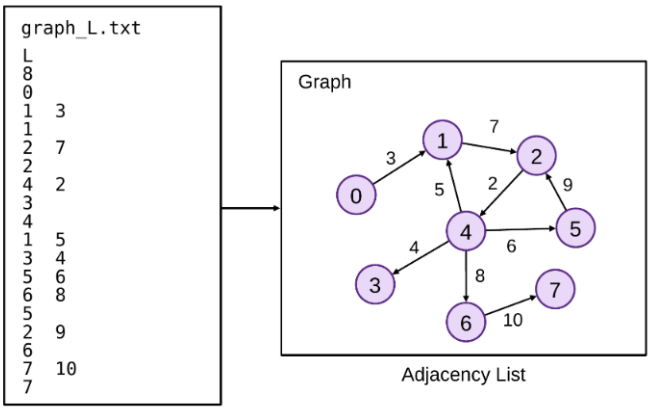


Figure 2. 인접 리스트 그래프 파일 구성

- 인접 행렬(adjacency matrix)은 **Figure 3** 과 같이 간선 정보가 N 행 N 열의 행렬 형태로 저장된다. 행(row)은 출발 정점, 열(column)은 도착 정점을 나타내며, 행렬의 값(value)은 해당 간선의 가중치를 의미한다. 간선이 없는 경우에는 0 을 저장하며, 자기 자신으로 연결되는 간선이 없으므로 대각 원소는 모두 0 이다.

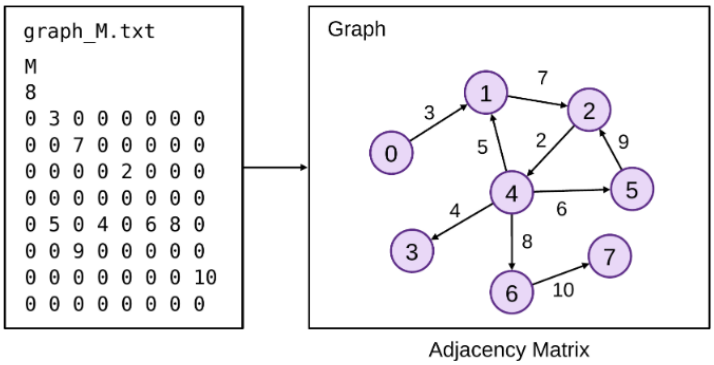


Figure 3. 인접 행렬 그래프 파일 구성

- 텍스트 파일에서 읽은 그래프 저장 유형과 정점 개수는 Graph 클래스에서 관리한다. 간선 연결 정보와 가중치는 저장 유형에 따라 ListGraph 또는 MatrixGraph 에 저장한다.

2) 그래프 연산

- BFS, DFS (방향성 O/X, 가중치 X)

1. BFS 및 DFS 명령어는 방향성 적용 여부만 고려하여 그래프 탐색을 수행한다.
2. 프로그램은 각 명령어(BFS O 1, DFS X 2)에 포함된 시작 정점에서 탐색하기 시작하여 방문 가능한 모든 정점을 방문하고, 정점 방문 순서를 로그 파일에 출력한다.
3. 그래프와 명령어의 구성에 따라 방문 되지 않는 정점이 존재할 수 있다.
4. 한 정점에서 방문할 수 있는 정점이 여러 개일 경우, 번호가 가장 낮은 정점을 먼저 방문한다고 가정한다.

- Kruskal (방향성 X, 가중치 O)

1. KRUSKAL 명령어는 방향성이 없고, 가중치만 고려하여 알고리즘을 수행한다.
2. 프로그램은 KRUSKAL 명령어 수행 시, 저장된 그래프 정보를 이용하여 그래프의 최소 신장 트리(Minimum Spanning Tree; MST)를 구하고, 이를 로그 파일에 출력한다.
3. 가중치 기준 간선 오름차순 정렬 시, 가중치가 같으면 두 끝점 중 작은 정점 번호를 먼저 비교하고, 이 번호도 같으면 큰 정점 번호를 비교하여 오름차순으로 처리한다.

- Dijkstra (방향성 O/X, 가중치 O)

1. DIJKSTRA 명령어는 방향성 적용 여부와 가중치를 모두 고려하여 알고리즘을 수행한다.
2. 프로그램은 DIJKSTRA 명령어(DIJKSTRA O 1)에 포함된 시작 정점에서 다른 모든 정점으로 가는 최단 경로(shortest path)와 그 비용(cost)을 로그 파일에 출력한다.
3. 저장된 그래프에 가중치가 음수인 간선이 존재하면 에러를 출력한다.

- Bellman-Ford (방향성 O/X, 가중치 O)

1. BELLMANFORD 명령어는 방향성 적용 여부와 가중치를 모두 고려하여 수행한다.
2. 프로그램은 BELLMANFORD 명령어에 포함된 출발 정점과 도착 정점 사이의 최단 경로와 비용을 로그 파일에 출력한다.

3. 저장된 그래프에 가중치가 음수인 간선이 존재해도 정상 동작하며, 알고리즘 수행 중 음수 사이클이 발생하면 에러를 출력한다.

- Floyd (방향성 O/X, 가중치 O)

1. FLOYD 명령어는 방향성 적용 여부와 가중치를 모두 고려하여 알고리즘을 수행한다.
2. 프로그램은 FLOYD 명령어 수행 시, 모든 정점 쌍에 대한 최단 경로 행렬을 구하고 로그 파일에 출력한다.
3. 저장된 그래프에 가중치가 음수인 간선이 존재해도 정상 동작하며, 알고리즘 수행 중 음수 사이클이 발생하면 에러를 출력한다.

- SORT (방향성 O/X, 가중치 O)

1. SORT [방향성] [vertex] [A/D] [H/M] 형식으로 입력하며, 지정한 정점의 인접 간선을 H 이면 힙 정렬, M 이면 병합 정렬로 정렬한다. 한 명령에서는 선택한 알고리즘 하나만 실행한다.
2. H 는 입력 배열, 힙 구성 직후 배열, 첫 추출 간선과 추출 후 배열, 최종 정렬 결과 및 각 단계의 누적 비교 횟수를 출력한다. M 은 입력 배열, 병합 단계 수, 단계별 너비, 배열, 누적 비교 횟수, 최종 결과 및 총 비교 횟수를 출력한다.
3. 정렬 전 간선 목록은 이웃 정점 번호 오름차순으로 구성하고, 비교 횟수는 실행마다 0 부터 계산한다.
4. 세 번째 인자가 A 이면 가중치 오름차순, D 이면 내림차순으로 정렬한다. 가중치가 같으면 두 경우 모두 이웃 정점 번호가 작은 간선을 먼저 출력한다.
5. H 는 마지막 내부 노드부터 루트까지 sift-down 하여 힙을 구성한다. 두 자식이 있으면 우선순위가 높은 자식을 선택한 뒤 부모와 비교한다. 루트를 추출한 뒤 마지막 원소를 루트로 옮기고 같은 방식으로 복구한다.
6. M 은 너비 1, 2, 4, … 순서의 반복형 병합 정렬을 사용한다. 각 구간 쌍을 왼쪽부터 병합하고, 두 원소의 우선순위가 같으면 왼쪽 원소를 먼저 선택한다.
7. 비교 횟수는 두 간선의 우선순위를 비교하는 함수 호출 한 번을 1 회로 센다. 가중치와 정점 번호를 함께 비교해도 1 회이며, 반복 조건, 인덱스 검사, 복사, 교환은 제외한다. 각 단계 수치는 실행 시작부터의 누적값이다.

☐ Functional Requirements

- 프로그램은 명령어를 통해 동작하며, 실행할 명령어가 작성되어 있는 텍스트 파일(command.txt)를 읽고 명령어에 따라 순차적으로 동작한다. 모든 명령어는 반드시 **대문자**로 입력하며, 명령어의 인자는 모두 **공백(' ') 즉, space 한번**으로 구분한다.

명령어	명령어 사용 예 및 기능
LOAD	사용 방법) LOAD [textfile] 예시) LOAD graph_L.txt (or graph_M.txt) - 프로그램에 그래프 정보를 불러오는 명령어로, 그래프 정보가 저장되어 있는 텍스트 파일(graph_L.txt 또는 graph_M.txt)을 읽어 그래프를 구성한다. - 여러 번 LOAD 하는 경우, 기존 그래프를 삭제하고 재구성한다. - 인자의 개수가 잘못되었거나, 파일명이 graph_L.txt 또는 graph_M.txt 가 아니거나, 파일을 읽을 수 없거나, 파일이 없을 경우 출력 파일에 에러 코드(100)를 출력한다. - 텍스트 파일: graph_L.txt or graph_M.txt (이름 변경 시, 프로젝트 0 점) 출력 포맷 예시) (1) LOAD 성공 시 ==========LOAD========== Success ======================== (2) error 발생 ==========ERROR========== 100 ========================
PRINT	사용 방법) PRINT 예시) PRINT - 그래프 정보를 출력하는 명령어로, List 형태로 저장된 경우, 인접 리스트(adjacency list)를, Matrix 형태로 저장된 경우, 인접 행렬 (adjacency matrix)을 출력 파일에 출력한다. - List 형태로 출력 시, 출발 정점은 오름차순으로 출력하고, 간선은 도착 정점 기준 오름차순으로 출력한다. - 저장된 그래프가 없거나 인자가 추가로 입력된 경우, 출력 파일에 에러 코드(200)를 출력한다. - 텍스트 파일: graph_L.txt or graph_M.txt (이름 변경 시, 프로젝트 0 점)

출력 포맷 예시)

(1)-1. PRINT

모두 양수 가중치를 가지는 경우

```
=========PRINT=========
[0] -> (1,3)
[1] -> (2,7)
[2] -> (4,2)
[3] ->
[4] -> (1,5) -> (3,4) -> (5,6) -> (6,8)
[5] -> (2,9)
[6] -> (7,10)
[7] ->
=======================

==========PRINT==========
    [0] [1] [2] [3] [4] [5] [6] [7]
[0]  0   3   0   0   0   0   0   0
[1]  0   0   7   0   0   0   0   0
[2]  0   0   0   0   2   0   0   0
[3]  0   0   0   0   0   0   0   0
[4]  0   5   0   4   0   6   8   0
[5]  0   0   9   0   0   0   0   0
[6]  0   0   0   0   0   0   0  10
[7]  0   0   0   0   0   0   0   0
=========================
```

(1)-2. PRINT

음수 가중치가 포함된 경우

```
=========PRINT=========
[0] -> (1,-3)
[1] -> (2,7)
[2] -> (4,2)
[3] ->
[4] -> (1,5) -> (3,4) -> (5,6) -> (6,8)
[5] -> (2,9)
[6] -> (7,10)
[7] ->
=======================
```

	==========PRINT========== [0] [1] [2] [3] [4] [5] [6] [7] [0] 0 -3 0 0 0 0 0 0 [1] 0 0 7 0 0 0 0 0 [2] 0 0 0 0 2 0 0 0 [3] 0 0 0 0 0 0 0 0 [4] 0 5 0 4 0 6 8 0 [5] 0 0 9 0 0 0 0 0 [6] 0 0 0 0 0 0 0 10 [7] 0 0 0 0 0 0 0 0 ======================== (2) error 발생 =========ERROR========= 200 ========================
BFS	사용 방법) BFS [방향성] [vertex] 예시 1) BFS O 1 예시 2) BFS X 1 - O 는 방향성이 있는 그래프를 나타내고, X 는 방향성이 없는 무방향 그래프를 나타낸다. - 마지막 인자는 시작 정점 번호이며, 시작 정점부터 BFS 를 수행하고 방문 순서를 출력 파일에 출력한다. - 그래프가 없거나, 시작 정점이 존재하지 않거나, 인자의 개수 및 형식이 잘못된 경우 출력파일에 에러 코드(300)를 출력한다. 출력 포맷 예시) (1) BFS O 1 ==========BFS========== Directed Graph BFS Start: 1 1 -> 2 -> 4 -> 3 -> 5 -> 6 -> 7 ======================== (2) BFS X 1 ==========BFS========== Undirected Graph BFS Start: 1 1 -> 0 -> 2 -> 4 -> 5 -> 3 -> 6 -> 7 ========================

	(3) error 발생 =========ERROR========= 300 =======================
DFS	사용 방법) DFS [방향성] [vertex] 예시 1) DFS O 1 예시 2) DFS X 1 - O 는 방향성이 있는 그래프를 나타내고, X 는 방향성이 없는 무방향 그래프를 나타낸다. - 마지막 인자는 시작 정점 번호이며, 시작 정점부터 DFS 를 수행하고 방문 순서를 출력 파일에 출력한다. - 그래프가 없거나, 시작 정점이 존재하지 않거나, 인자의 개수 및 형식이 잘못된 경우 출력파일에 에러 코드(400)를 출력한다. 출력 포맷 예시) (1) DFS O 1 ==========DFS========== Directed Graph DFS Start: 1 1 -> 2 -> 4 -> 3 -> 5 -> 6 -> 7 ======================== (2) DFS X 1 ========DFS======== Undirected Graph DFS Start: 1 1 -> 0 -> 2 -> 4 -> 3 -> 5 -> 6 -> 7 ====================== (3) error 발생 =========ERROR========= 400 =======================
KRUSKAL	사용 방법) KRUSKAL - 저장된 그래프의 최소 신장 트리(MST)를 구하며, MST 를 구성하는 간선들의 정점 번호 및 가중치를 정점 번호 오름차순으로 출력하고, 가중치의 총합을 출력 파일에 출력한다. 방향성은 없다.

	- 그래프가 없거나, 인자가 추가로 입력되거나, 접근 불가능한 정점이 있는 경우, 모든 정점을 연결하는 최소 신장 트리를 구성할 수 없는 경우 출력파일에 에러 코드(500)를 출력한다. 출력 포맷 예시) (1) KRUSKAL ========KRUSKAL======== [0] 1(3) [1] 0(3) 4(5) [2] 4(2) [3] 4(4) [4] 1(5) 2(2) 3(4) 5(6) 6(8) [5] 4(6) [6] 4(8) 7(10) [7] 6(10) Cost: 38 ========== (2) error 발생 =========ERROR========= 500 ==========
DIJKSTRA	사용 방법) DIJKSTRA [방향성] [vertex] 예시 1) DIJKSTRA O 1 예시 2) DIJKSTRA X 1 - O 는 방향성이 있는 그래프를 나타내고, X 는 방향성이 없는 무방향 그래프를 나타낸다. - 마지막 인자는 시작 정점 번호이며, 시작 정점부터 Dijkstra 알고리즘을 수행하여 시작 정점을 포함한 다른 모든 정점까지의 최단 경로(shortest path)와 그 비용(cost)을 출력 파일에 출력한다. - 경로는 시작 정점부터 도착 정점까지 순서대로 출력한다. - 시작 정점에서 도착 정점에 도달할 수 없는 경우, 최단 경로와 비용 대신 x 를 출력한다. - 그래프가 없거나, 시작 정점이 존재하지 않거나, 저장된 그래프에 음의 가중치 간선이 있거나, 인자의 개수 및 형식이 잘못된 경우 출력파일에 에러 코드(600)를 출력한다. 출력 포맷 예시) 예시 1) DIJKSTRA O 1

	========DIJKSTRA======== Directed(or Undirected) Graph Dijkstra ← 방향성에 따라 다르게 출력 Start: 1 [0] x [1] 1 (0) [2] 1 -> 2 (7) [3] 1 -> 2 -> 4 -> 3 (13) [4] 1 -> 2 -> 4 (9) [5] 1 -> 2 -> 4 -> 5 (15) [6] 1 -> 2 -> 4 -> 6 (17) [7] 1 -> 2 -> 4 -> 6 -> 7 (27) ======================== (2) error 발생 =========ERROR========= 600 ========================
BELLMANFORD	사용 방법) BELLMANFORD [방향성] [vertex1] [vertex2] 예시 1) BELLMANFORD O 4 2 예시 2) BELLMANFORD X 4 2 - O 는 방향성이 있는 그래프를 나타내고, X 는 방향성이 없는 무방향 그래프를 나타낸다. - [vertex1] 인자는 시작 정점 번호이며, [vertex2] 인자는 도착 정점 번호이다. - 시작 정점부터 도착 정점까지의 최단 경로와 비용을 Bellman-Ford 알고리즘을 수행하여 계산하고, 최단 경로와 그 비용을 출력 파일에 출력한다. - 시작 정점에서 도착 정점에 도달할 수 없는 경우 경로와 비용 대신 x 를 출력한다. - 그래프가 없거나, 시작 또는 도착 정점이 존재하지 않거나, 시작 정점으로부터 도달 가능한 음수 사이클이 있거나, 인자의 개수 및 형식이 잘못된 경우 출력파일에 에러 코드(700)를 출력한다. 출력 포맷 예시) (1) BELLMANFORD O 4 2 ========BELLMANFORD======== Directed(or Undirected) Graph Bellman-Ford ← 방향성에 따라 다르게 출력 4 -> 1 -> 2 Cost: 12 ======================== (2) error 발생 =========ERROR========= 700

	=======================
FLOYD	사용 방법) FLOYD [방향성] 예시 1) FLOYD O 예시 2) FLOYD X - O 는 방향성이 있는 그래프를 나타내고, X 는 방향성이 없는 무방향 그래프를 나타낸다. - 저장된 그래프의 모든 정점 쌍에 대한 최단 경로의 비용을 행렬 형태로 출력한다. - 행렬의 행은 시작 정점이고, 열은 도착 정점이다. - 시작 정점에서 도착 정점에 도달할 수 없는 경우 비용 대신 x 를 출력한다. - 그래프가 없거나, 선택한 방향성에서 음수 사이클이 있거나, 인자의 개수 및 형식이 잘못된 경우 출력파일에 에러 코드(800)를 출력한다. 출력 포맷 예시) (1) FLOYD O ============FLOYD============ Directed(or Undirected) Graph Floyd ← 방향성에 따라 다르게 출력 [0] [1] [2] [3] [4] [5] [6] [7] [0] 0 3 10 16 12 18 20 30 [1] x 0 7 13 9 15 17 27 [2] x 7 0 6 2 8 10 20 [3] x x x 0 x x x x [4] x 5 12 4 0 6 8 18 [5] x 16 9 15 11 0 19 29 [6] x x x x x x 0 10 [7] x x x x x x x 0 ============================ (2) error 발생 =========ERROR========= 800 =======================
SORT	사용 방법) SORT [방향성] [vertex] [A/D] [H/M] 예시 1) SORT O 4 A H 예시 2) SORT O 4 D M - 지정한 정점의 인접 간선을 H 이면 힙 정렬, M 이면 병합 정렬로 정렬하며, 선택한 알고리즘의 정렬 전 배열, 중간 과정, 최종 정렬 결과 및 비교 횟수를 출력한다. - 첫 번째 인자는 방향성(O/X), 두 번째 인자는 정점 번호, 세 번째 인자는 정렬 방향(A: 오름차순, D: 내림차순)이다. 네 번째 인자는 정렬 방법(H: 힙 정렬, M: 병합 정렬)이다.

	- 간선은 (연결된 정점, 가중치) 형태로 출력하며, 정렬할 간선이 없으면 에러 코드(900)를 출력한다. SORT 는 저장된 그래프를 변경하지 않는다. - 그래프가 없거나, 지정한 정점이 존재하지 않거나, 선택한 방향성에서 정렬할 인접 간선이 없거나, 인자의 개수 및 형식이 잘못된 경우 출력파일에 에러 코드(900)를 출력한다. 출력 포맷 예시) (1) SORT O 4 A H ========SORT======== Directed(or Undirected) Graph SORT ← 방향성에 따라 다르게 출력 Start: 4 Order: A Method: H Input: (1,5) -> (3,4) -> (5,6) -> (6,8) HeapBuild: (3,4) -> (1,5) -> (5,6) -> (6,8) HeapBuildComparisons: 4 HeapFirstPop: (3,4) HeapAfterFirstPop: (1,5) -> (6,8) -> (5,6) HeapFirstPopComparisons: 6 Heap: (3,4) -> (1,5) -> (5,6) -> (6,8) HeapComparisons: 7 ========== 예시 2) SORT O 4 D M ========SORT======== Directed(or Undirected) Graph SORT ← 방향성에 따라 다르게 출력 Start: 4 Order: D Method: M Input: (1,5) -> (3,4) -> (5,6) -> (6,8) MergePasses: 2 MergeWidth: 1 Comparisons: 2 MergeData: (1,5) -> (3,4) -> (6,8) -> (5,6) MergeWidth: 2 Comparisons: 4 MergeData: (6,8) -> (5,6) -> (1,5) -> (3,4) Merge: (6,8) -> (5,6) -> (1,5) -> (3,4) MergeComparisons: 4 ========== (3) error 발생 =========ERROR========= 900 ==========
EXIT	사용 방법) EXIT - 프로그램상의 모든 메모리를 해제하며, 프로그램을 종료한다. 오류가 발생하는 상황은 없다.

	출력 포맷 예시) ========EXIT======== Success ====================

☐ 동작 별 에러 코드

동작	에러 코드
LOAD	100
PRINT	200
BFS	300
DFS	400
KRUSKAL	500
DIJKSTRA	600
BELLMANFORD	700
FLOYD	800
SORT	900
잘못된 명령어	1000

☐ Requirements in implementation

- 모든 명령어는 command.txt 에 저장하여 순차적으로 읽고 처리한다.
- 모든 명령어와 옵션은 반드시 대문자로 입력한다.
- 명령어에 인자(Parameter)가 모자라거나 필요 이상으로 입력 받으면 에러 코드를 출력한다.
- 예외처리에 대해 반드시 에러 코드를 출력한다.
- 메모리 누수 발생 시, 감점 10%
- 잘못된 출력 시, 기능 별 감점
- 출력은 "출력 포맷"을 반드시 따라 한다.
- log.txt 파일에 출력 결과를 반드시 저장한다.
  - log.txt 가 이미 존재할 경우 텍스트 파일 가장 뒤에 이어서 추가로 저장한다.

☐ STL 사용 규정

- STL 은 파일,문자열 입출력 스트림, 기본 자료형 및 오류 처리를 위한 표준 라이브러리는 사용할 수 있다. 그 외에는 vector, stack, queue, string, pair 만 사용할 수 있으며, map 은 그래프의 인접 정보 저장 용도로만 사용한다.
- SORT, KRUSKAL 명령어의 Heap 은 vector 를 이용해 직접 구현하며, priority_queue 와 make_heap, push_heap, pop_heap, sort_heap 등 heap 관련 algorithm 함수와 sort, stable_sort, qsort 등 정렬 함수는 사용하지 않는다. 병합 정렬도 vector 를 이용하여 직접 구현하며, merge 와 inplace_merge 등 정렬 과정을 대신하는 라이브러리 함수는 사용하지 않는다.

□ 구현 시 반드시 정의해야하는 Class 및 파일

- Graph : Graph 클래스
- ListGraph : ListGraph 클래스
- MatrixGraph : MatrixGraph 클래스
- GraphMethod : 그래프 연산을 수행하는 함수를 모아둔 파일
- Manager : Manager 클래스,
  다른 클래스들의 동작을 관리하여 프로그램을 전체적으로 조정하는 역할을 수행

□ Files

- graph_L.txt : 그래프가 인접 리스트 형태로 저장된 파일
- graph_M.txt : 그래프가 인접 행렬 형태로 저장된 파일
- command.txt : 프로그램을 동작시키는 명령어들을 저장하고 있는 파일
- log.txt : 프로그램 출력 결과를 모두 저장하고 있는 파일

□ 제한사항 및 구현 시 유의사항

- 반드시 제공되는 코드(github 주소 참고)를 이용하여 구현하며 작성된 소스 파일의 이름과 클래스와 함수 이름 및 형태를 임의로 변경하지 않는다.
  - 클래스의 함수 및 변수는 자유롭게 추가 구현이 가능하다.
  - 제시된 Class 를 각 기능에 알맞게 모두 사용한다.
  - 프로그램 구조에 대한 디자인이 최대한 간결하도록 고려하여 설계한다.
  - 채점 시 코드를 수정해야 하는 일이 없도록 한다.
  - 주석은 반드시 영어로 작성한다. (한글로 작성하거나 없으면 감점)
  - 프로그램은 반드시 리눅스(Ubuntu 20.04)에서 동작해야 한다. (컴파일 에러 발생 시 감점)
    - 제공되는 Makefile 을 사용하여 테스트하도록 한다.
- 인터넷에서 공유된 코드, AI 로 생성된 코드나 다른 학생이 작성한 코드를 절대 카피하지 않도록 하며, 적발 시 해당 프로젝트 0 점 처리됨을 명시한다.

□ 채점기준

✓ 코드

채점 기준	점수
LOAD(0.5 점)와 PRINT(0.5 점) 명령어가 정상 동작하는가?	1
BFS(1 점)와 DFS(1 점) 명령어가 정상 동작하는가?	2
DIJKSTRA(1 점), KRUSKAL(1.5 점), BELLMANFORD(1.5 점) 명령어가 정상 동작하는가?	4
FLOYD(1 점)와 SORT(2 점)명령어가 정상 동작하는가?	3
총합	**10**

– 채점 기준 이외에도 조건 미달 시 감점 (linux 컴파일 에러, 파일 입출력 X, 주석 미흡 등)

□ 제출기한 및 제출방법

✓ 제출 기한

- 2026 년 12 월 04 일 금요일 23:59:59 까지 클라스(KLAS)에 제출
- (추가 제출: 2026 년 12 월 05 일 토요일 00:59:59 까지, 10% 감점)

✓ 제출 방법

- 소스코드를 함께 압축하여 제출
- 확장자가 .cpp, .h 가 아닌 파일은 제출하지 않음 (그 외 확장자 파일 제출 시 감점)

✓ 제출 형식

- 학번_DS_project3.tar.gz (ex. 2026XXXXXX_DS_project3.tar.gz)
  - 제출 형식을 지키지 않은 경우 감점